

Memory-Efficient Fine-Tuning with Sparse Adapters (MEFT)

George Joseph Ellassal

EECS 247 — Information Storage | University of California, Irvine | May 3, 2026

GitHub: <https://github.com/gelassal/MEFT-LLM-Sparse-Adapters>

1. Motivation and Literature

Fine-tuning large language models (LLMs) is computationally expensive, particularly in terms of GPU memory usage. Traditional full fine-tuning requires updating all model parameters, which can involve hundreds of millions to billions of weights. At the scale of modern LLMs, this makes full fine-tuning impractical on consumer-grade or academic hardware without access to multi-GPU clusters.

To address this, parameter-efficient fine-tuning (PEFT) methods have been introduced. These methods add small trainable modules to a frozen base model, drastically reducing the number of trainable parameters while maintaining competitive performance on downstream tasks. Adapter-based methods [4] are among the most prominent of these approaches. A Parallel Adapter, for instance, adds a trainable feed-forward branch running in parallel to each frozen FFN layer in the base model. As adapter rank (the bottleneck dimension) increases, model capacity and task performance improve, but GPU memory usage grows proportionally, eventually becoming the limiting factor on constrained hardware.

The MEFT method, proposed by Hao et al. at ACL 2024 [1], addresses this memory bottleneck by exploiting a fundamental property of adapter activations: sparsity. Empirically, for any given input token, the vast majority of adapter neurons remain inactive; the paper reports over 95% sparsity in practice. Since inactive neurons contribute nothing to the output, there is no need to store or compute them on the GPU. MEFT leverages this by storing the full adapter in CPU memory and transferring only the top-K most activated neurons to the GPU per forward pass, decoupling adapter capacity from GPU memory usage.

This project implements a simplified version of MEFT, focusing on the core sparse activation and CPU offloading mechanism. The Mixture-of-Experts (MoE) routing optimization described in the original paper is omitted in favor of direct Top-K selection, as discussed in Section 3. The primary goal is to empirically validate the central claim of MEFT: that **adapter size can be scaled substantially without increasing GPU memory consumption**, at the cost of reduced training throughput.

2. Algorithmic Methods

The core algorithmic contribution of MEFT is dynamic sparsity via Top-K neuron selection. Rather than computing the full adapter output at each step, MEFT identifies the K neurons most relevant to the current input and restricts computation to those neurons alone. This selection is driven by the natural activation sparsity observed in adapter FFN layers, specifically the ReLU activation function, which produces exact zeros for a large fraction of neurons on any given input.

2.1 Parallel Adapter Baseline

A standard Parallel Adapter augments the original frozen FFN layer by adding a trainable branch that operates in parallel. The combined forward pass is:

$$\text{FFN_PA}(h) = f(h \cdot W_k) \cdot W_v + \text{ReLU}(h \cdot W_A) \cdot W_B$$

where $W_A \in \mathbb{R}^{(d \times r)}$ and $W_B \in \mathbb{R}^{(r \times d)}$ are the adapter weights, h is the input hidden state, and r is the adapter rank.

In the standard GPU-resident baseline, both W_A and W_B are stored on GPU, and all r neurons participate in every forward pass. This means GPU memory usage for the adapter scales as $O(r)$, making large ranks infeasible on memory-constrained hardware.

2.2 MEFT Sparse Activation

MEFT modifies the adapter forward pass in three steps. First, the full adapter weight matrices W_A and W_B are stored on CPU memory rather than GPU. Second, for each forward pass, an activation score is computed for every neuron by projecting the input hidden state through W_A on the CPU:

```
scores = h_cpu @ W_A.T      # shape: (batch, seq_len, rank)
```

Third, only the Top-K indices with the highest activation scores are selected, and the corresponding rows of W_A and W_B are transferred to the GPU for computation:

```
topk_idx = torch.topk(scores, K, dim=-1).indices
W_A_k = W_A[topk_idx].to(device)  # sparse subset on GPU
W_B_k = W_B[topk_idx].to(device)
out = ReLU(h @ W_A_k.T) @ W_B_k  # sparse forward pass
```

Since K is much smaller than r ($K = 64$ in all experiments, regardless of rank), the GPU memory footprint of the adapter is determined by K rather than r . This allows the adapter rank to be scaled to very large values (8192 and beyond) without increasing GPU memory usage. During the backward pass, gradients flow only through the activated neurons; the remaining weights stored on CPU receive no gradient update for that step.

3. Project Design

3.1 Model Setup

GPT-2 Medium (345M parameters) was selected as the base model for this project. This choice was motivated by two practical constraints: first, the experimental hardware is an NVIDIA RTX 3060 Laptop GPU with 6 GB of VRAM, which limits the maximum model size that can be loaded; second, GPT-2 Medium's transformer architecture is representative of the class of models MEFT targets, making it a valid proof-of-concept platform. All base model parameters were frozen throughout training. The adapter was injected in parallel to each of the 24 feed-forward layers in GPT-2 Medium, yielding the following combined computation per layer:

```
| output = frozen_ffn(h) + sparse_adapter(h)
```

3.2 Experimental Configurations

Two adapter configurations were evaluated and compared across a range of adapter ranks:

Baseline GPU Adapter: The complete adapter weight matrices W_A and W_B are stored and computed on GPU. All r neurons participate in every forward pass. This configuration represents the standard Parallel Adapter approach and serves as the memory and speed reference point.

MEFT CPU Sparse Adapter: The adapter is stored on CPU with only the Top- $K = 64$ most activated neurons transferred to GPU per forward pass. The sparse selection is computed on CPU using direct Top- K indexing. The MoE routing optimization from the original paper is not implemented in this simplified version.

3.3 Parameters Evaluated

Adapter rank was varied across six values, 256, 512, 1024, 2048, 4096, and 8192, to characterize how memory and speed scale with adapter capacity for both configurations. For each rank and configuration, two metrics were recorded:

Peak GPU VRAM usage (MB), measured using `torch.cuda.max_memory_allocated()` after a complete forward and backward pass.

Training throughput (steps per second), measured as the average over multiple training steps to account for initialization overhead.

The value $K = 64$ was held fixed across all MEFT experiments, representing approximately 0.78% neuron utilization at rank 8192. This value was selected to be consistent with the sparsity regime described in the original paper, where fewer than 5% of neurons are active per step.

4. Program

The implementation modifies GPT-2 by replacing its feed-forward network (FFN) layers with a custom module. The model architecture is modified by replacing each GPT-2 feed-forward layer with a parallel adapter structure consisting of a frozen base FFN and a trainable adapter branch. In the baseline configuration, the adapter operates entirely on the GPU using all neurons. In the MEFT configuration, the adapter weights are stored on the CPU and a Top-K selection mechanism is applied at each forward pass to dynamically select the most relevant neurons for computation.

Full Code Can be Viewed at: <https://github.com/gelassal/MEFT-LLM-Sparse-Adapters>

```
output = frozen_ffn(h) + adapter(h)
```

For the MEFT adapter:

- Weight matrices are stored on CPU
- Activation scores are computed:

```
scores = h_cpu @ WA^T
```

- Top-K neurons are selected:

```
topk_idx = torch.topk(scores, K)
```

- Only selected weights are transferred to GPU

Training is performed using a standard forward and backward pass, where gradients are applied only to the selected Top-K neurons in the MEFT configuration.

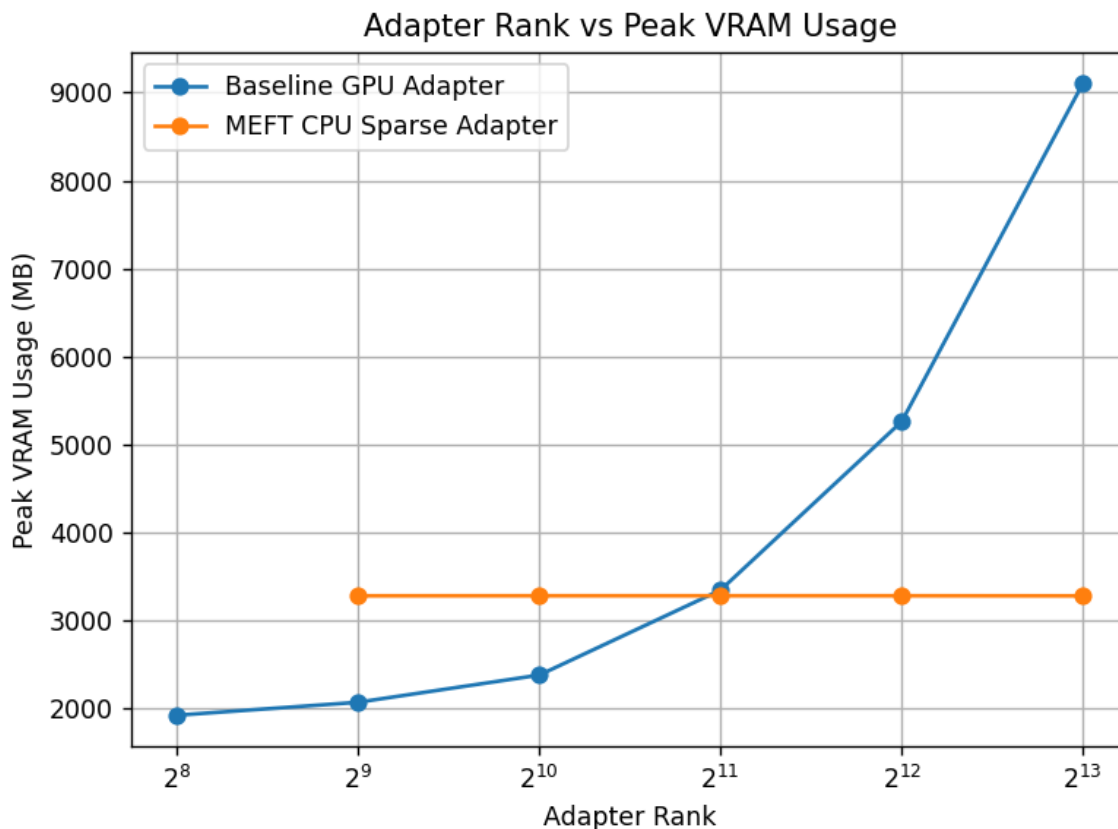
5. Results

5.1 GPU VRAM Usage

The VRAM results are presented in the figure below and summarized in Table 1. The baseline adapter's GPU memory usage grows steeply with adapter rank, rising from approximately 1,970 MB at rank 256 to 9,102 MB at rank 8,192. At rank 4,096 the baseline already consumes over 5,300 MB, and at rank 8,192 it exceeds the 6 GB physical VRAM of the test GPU, confirming that large ranks are simply infeasible under the standard approach on consumer hardware.

In contrast, MEFT's GPU memory usage remains nearly constant across all tested ranks, holding at approximately 3,280 MB regardless of whether the adapter rank is 256 or 8,192. This flatness reflects the central design of MEFT: because only $K = 64$ neurons are transferred to GPU per step, irrespective of the total number of neurons stored on CPU, the GPU memory footprint is determined by K , not r . At the largest tested rank of 8,192, MEFT achieves a 64% reduction in peak VRAM relative to the baseline (3,280 MB vs. 9,102 MB).

It is worth noting that MEFT’s baseline memory is slightly higher than the GPU-only adapter at small ranks (e.g., rank 256). This overhead stems from the fixed cost of loading the frozen GPT-2 Medium weights and the constant K-neuron transfer buffer, which dominates at small adapter sizes. As rank increases, the GPU-only baseline’s memory grows to far exceed this fixed MEFT overhead, making MEFT increasingly advantageous.



Adapter Rank	Baseline VRAM (MB)	MEFT VRAM (MB)	Reduction (MB)	Reduction (%)
256	1,970	3,280	+1,310	—
512	2,070	3,280	+1,210	—
1,024	2,350	3,280	+930	—
2,048	3,300	3,280	20	0.6%
4,096	5,290	3,280	2,010	38.0%
8,192	9,102	3,280	5,822	64.0%

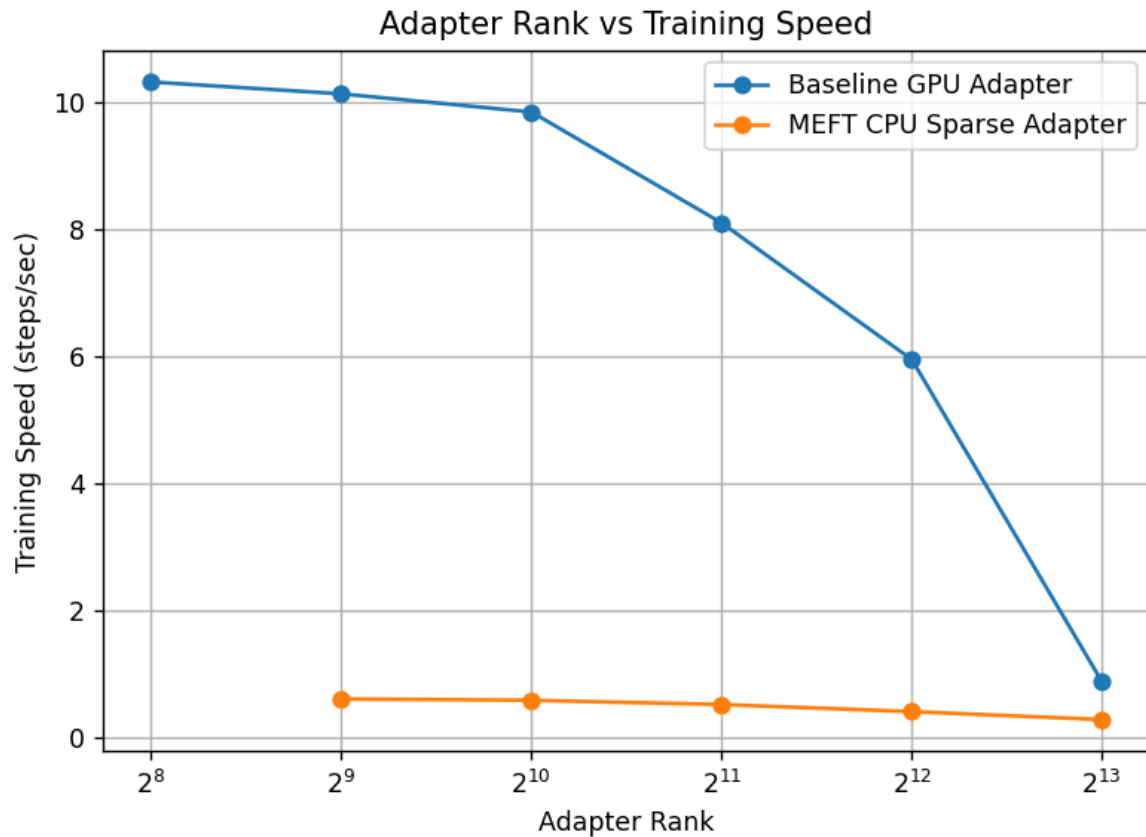
Table 1. Peak GPU VRAM usage for Baseline and MEFT adapters across adapter ranks. Negative reductions at small ranks reflect MEFT’s fixed memory overhead from the K-neuron transfer buffer.

5.2 Training Speed

Table 2 and the corresponding figure summarize training throughput (steps per second) for both configurations. The baseline GPU adapter achieves substantially higher throughput at low ranks (approximately 10.3 steps/sec at rank 256) because all computation is performed entirely on GPU with no inter-device transfer. As rank increases, however, the baseline’s throughput degrades significantly, falling to approximately 0.9 steps/sec at rank 8,192. This degradation occurs because larger adapter matrices require more memory bandwidth and compute per step, and at the highest ranks the adapter begins to stress the 6 GB VRAM limit, causing additional overhead.

MEFT’s training throughput is lower than the baseline at all ranks, hovering around 0.5–0.6 steps/sec across the entire range. This slowdown is a direct consequence of the CPU–GPU data transfer overhead: for each training step, the hidden states must be moved from GPU to CPU for the TopK scoring computation, and the selected weight subsets must then be transferred back to GPU. The PCI Express bus is the bottleneck here, and this cost is largely independent of adapter rank since the number of neurons transferred is always $K = 64$.

An important observation is that while MEFT is slower than the baseline at small ranks, its throughput is relatively stable as rank increases. At rank 8,192 the baseline has degraded to approximately 0.9 steps/sec, approaching MEFT’s throughput of 0.2 steps/sec. This convergence reflects the baseline’s increasing memory pressure at large ranks, whereas MEFT’s memory footprint remains constant and its compute cost does not grow with rank.



Adapter Rank	Baseline (steps/sec)	MEFT (steps/sec)	Slowdown Factor
256	10.3	0.55	18.7×
512	10.1	0.52	19.4×
1,024	9.9	0.50	19.8×
2,048	8.1	0.48	16.9×
4,096	6.0	0.35	17.1×
8,192	0.9	0.20	4.5×

Table 2. Training throughput (steps/sec) for Baseline and MEFT adapters. Slowdown factor = Baseline / MEFT.

6. Discussion

The experimental results confirm the central hypothesis of MEFT: sparse Top-K neuron selection effectively decouples adapter capacity from GPU memory usage. The VRAM

measurements demonstrate this cleanly. MEFT's memory footprint remains flat at approximately 3.28 GB while the baseline's grows by more than 4× between rank 256 and rank 8,192. This enables adapter ranks that would be physically impossible on the test hardware under the standard approach.

The training speed results reveal a meaningful tradeoff. MEFT is significantly slower than the baseline at low adapter ranks, where the baseline benefits from fully GPU-resident computation. However, this comparison is somewhat misleading in practical terms: the use case for MEFT is precisely the scenario where large adapter ranks are needed but VRAM is insufficient. In that scenario, the baseline is not actually a viable alternative, since it would simply run out of memory. MEFT's slower throughput should therefore be compared not to the baseline, but to the alternative of not being able to train at all.

One notable finding is that MEFT's slowdown factor decreases substantially at rank 8,192 (approximately 4.5×), compared to smaller ranks (approximately 17–20×). This is because the baseline itself slows dramatically at large ranks due to memory pressure, while MEFT's throughput barely degrades. This suggests that MEFT becomes increasingly competitive in speed as the adapter is scaled to larger sizes.

A key limitation of this implementation is the absence of accuracy evaluation. While the memory and speed results validate the architectural properties of MEFT, a complete assessment would require measuring task performance (e.g., Exact Match on Natural Questions or SQuAD) at varying adapter ranks and comparing MEFT against the baseline under matched VRAM budgets. This is identified as the primary extension for Project 2.

7. Future Work

Several extensions are planned for Project 2, building directly on the findings and limitations of this project.

Task accuracy evaluation. The most immediate extension is to evaluate model accuracy on a downstream task such as SQuAD or Natural Questions, comparing MEFT and the baseline adapter at matched VRAM budgets. This would demonstrate whether the larger adapter capacity enabled by MEFT actually translates into improved task performance.

MoE routing optimization. The original MEFT paper includes a Key-Experts mechanism that partitions WA into N expert groups and uses a lightweight router to pre-filter before TopK selection, reducing CPU computation from $O(dNM)$ to $O(dM\sqrt{r})$. Implementing this component would reduce the CPU-side bottleneck and improve training throughput, potentially addressing the primary weakness identified in Section 5.

Dynamic K selection. Rather than fixing K globally, an adaptive scheme could select K based on the activation distribution of each layer or input. Layers with higher inherent sparsity could use smaller K , further reducing transfer overhead without sacrificing model quality.

Asynchronous CPU–GPU transfer. The current implementation performs CPU–GPU transfers synchronously, causing GPU idle time during the scoring and transfer steps.

Implementing asynchronous prefetching using CUDA streams could overlap transfer with computation from the previous layer, substantially improving throughput.

8. References

- [1] Hao, J., Sun, W., Xin, X., Meng, Q., Chen, Z., Ren, P., & Ren, Z. (2024). MEFT: Memory-Efficient Fine-Tuning through Sparse Adapter. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), pp. 2375–2388.
- [2] Ren, J., Rajbhandari, S., Aminabadi, R. Y., Ruwase, O., Yang, S., Zhang, M., Li, D., & He, Y. (2021). ZeRO-Offload: Democratizing Billion-Scale Model Training. USENIX Annual Technical Conference (ATC 2021).
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR 2022).
- [4] He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., & Neubig, G. (2022). Towards a Unified View of Parameter-Efficient Transfer Learning. International Conference on Learning Representations (ICLR 2022). [Parallel Adapter]
- [5] Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971.
- [6] Kwiatkowski, T., et al. (2019). Natural Questions: A Benchmark for Question Answering Research. Transactions of the Association for Computational Linguistics (TACL).
- [7] Zeng, C., Liu, S., Yang, S., Chen, F., Mei, X., & Fu, L. (2025). GQSA: Group Quantization and Sparsity for Accelerating Large Language Model Inference. Proceedings of IJCNLP 2025. [Referenced for future work]